CASE STUDY

2WIN Alerts

Backend Engineering / Event Processing / Subscription Messaging

OVERVIEW

2WIN Alerts connects radio-competition callbacks to subscriber messaging. Two Python services coordinate incoming events, audio-based outcome analysis, subscriber records, and personalized SMS submission. Splash Cash is the active competition workflow in the documented implementation.

THE PROBLEM

A competition alarm and its eventual outcome arrive at different times. Subscribers need an initial prompt before the result is known. The application separates that first alert from the slower audio-analysis path, while managing paid-service records and recipient selection.

MY ROLE

I developed both Flask services and their integrations: the Info Server for callback intake and competition processing, and the Messaging Server for subscriber lookup, queued messaging, and subscription workflows. The project also includes integrations with audio-analysis APIs, Redis, Airtable, Stripe, and notification providers.

WORKFLOW

A Splash Cash callback submits an immediate alert request. Audio capture and transcript analysis then attempt to identify the outcome. When the Info Server receives a result within its wait window, it submits a second message request to the Messaging Server.

SYSTEM DESIGN

From callback to message

Two services coordinating event processing and subscriber messaging

EVENT INTAKE

The Info Server accepts competition and alarm data through a callback route and starts processing in a thread. A Redis key suppresses repeated processing for the same competition for 180 seconds. Only Splash Cash has an active handler in this baseline.

AUDIO ANALYSIS

A daemon detector captures configured radio-stream audio in FFmpeg chunks, sends it to a Whisper-compatible transcription API, and uses a GPT-compatible API to classify the transcript. The detector writes its result to Redis; it does not send SMS directly.

MESSAGING

The Messaging Server queries Airtable for active subscribers, caches contacts in Redis, and queues a message request. A worker formats personalized messages and submits SMS through Textlocal. Airtable holds customer, service, and outbound-message metadata.

EVENT PATH

Callback > Info Server > Immediate message request
Audio capture > Transcription > Outcome classification
Redis result > Info Server > Outcome message request
Messaging Server > Redis queue > Worker > SMS API

SUBSCRIPTIONS

A signed Stripe checkout webhook maps a purchase to a competition and writes service records to Airtable. Welcome messages and time-based expiry checks provide subscription lifecycle paths. SMTP email is used by lifecycle code; competition alerts are configured for SMS.

ENGINEERING

Decisions and outcomes

Implemented capabilities and the remaining reliability work

TIMING

The initial message request precedes audio analysis. The Info Server waits up to five minutes for a Redis outcome, while detection can run longer. A timeout sends no outcome request, and the result key is not reset before a new run. These remain handoff limitations.

DELIVERY

Redis queueing and worker heartbeats coordinate message processing, but one queue item processes only one calculated contact batch. Remaining-contact recovery is incomplete. Partial provider success can mark a whole batch processed, so the implementation does not establish reliable delivery to every subscriber.

OUTCOME

The code connects callback intake, audio analysis, subscriber lookup, queueing, and SMS submission, alongside checkout and expiry workflows. Provider delivery, deployment, usage, and performance have not been independently verified. Other competition processors remain dormant in the selected baseline.

TECHNOLOGY

Services: Python, Flask
Coordination and records: Redis, Airtable
Subscriptions and messaging: Stripe, Textlocal, SMTP
Audio: FFmpeg, transcription and chat-completion APIs
Deployment configuration: GitHub Actions, systemd, Nginx

EVIDENCE

Source baseline: main at cafb828. Key modules include handle_comp.py, splash_cash_detector.py, pipeline.py, worker.py, and notifications.py. Further work includes callback/contact access controls, recipient fallback rules, expiry-template rendering, and dashboard route alignment.